

Research Project: Implementation of an In-Memory Key-Value Store Based on Optimized B+ Trees

Janne Gruhler, Maxim Bellm, Albert Rico Martin, Matthias Raba

Department of Computer Science

Hochschule Furtwangen

Furtwangen, Germany

Abstract—B⁺-trees have historically been considered inferior to hash-based structures for in-memory workloads. Recent work argues that this gap is largely an artifact of implementation. We converted the reference implementation into a reusable static library and attempted to reproduce its benchmarks. We also built a Redis-compatible key-value store and compared it against Redis under YCSB2 Workload C. Our store becomes competitive beyond roughly 500 000 keys and outperforms Redis on range scans. Absolute throughput in our reproduction deviates from the published values, but qualitative trends are preserved.

Index Terms—B-Trees, In-Memory Database, Key-Value Store, Memory Layout

I. INTRODUCTION AND MOTIVATION

B⁺-trees are among the most fundamental index structures in database systems, but were long considered inferior for in-memory use. Systems such as Redis rely on hash-based structures with near-constant latency per access. The paper “*B-trees are back: Engineering fast and pageable node layouts*” by Müller, Benson and Leis [1] shows this is primarily an implementation artifact: through multiple node types, fingerprinting, hinting, prefix compression and SIMD acceleration, B⁺-trees match or exceed the throughput of the Adaptive Radix Tree (ART) and the Height Optimized Trie (HOT), while preserving key order at no additional cost. If this holds, the preference for hash-based indexes in in-memory systems is a matter of implementation history rather than on a structural property, making an ordered index viable where an unordered one is normally assumed.

II. ANALYSIS AND SETUP OF THE EXISTING REPOSITORY

The code is publicly available on GitHub but was tightly coupled to one specific compiler environment: SIMD intrinsics were embedded directly as AVX2 code, the build system was a hand-written Makefile, and dependencies like TLX had to be placed into the source tree by hand. In this state the code is difficult to build outside the authors’ environment. We restructured the repository into a static library (`libbtree`): the build system was migrated to CMake, the AVX2 intrinsics were moved behind a compiler-independent abstraction, and the code was updated to compile cleanly under current compilers. The library exposes a unified `DataStructureWrapper` interface and can be included in other projects via CMake. The abstraction, in theory, also allows the code to be compiled for architectures other than x86, although performance on such systems is untested.

III. REPRODUCTION OF BENCHMARK RESULTS

The benchmarks are driven by Python scripts that invoke benchmarking programs with parameters (key count, operation count, batch size), store results as CSV, and visualize them in R. Datasets included dense and sparse integer keys, Wikipedia article titles and randomly generated URLs. All measurements ran on a university VM provisioned with 4 vCPUs and 64 GB of RAM, hosted on an AMD EPYC 7702P. Each core has 64 KiB of L1 and 512 KiB of L2 cache, alongside 64 MiB of shared L3; the system runs Debian 13 with kernel 7.0.4. The benchmark workload is synchronous and single-threaded and therefore executes on one core; dedicated cores and a single NUMA node rule out contention from co-located tenants. This setup nevertheless differs from the bare-metal machine of the original publication, which must be kept in mind when comparing absolute numbers. We compiled with a newer compiler version but kept the original optimization and architecture flags unchanged.

Reproduction proved considerably more difficult than expected. Datasets are only described in prose and not included in the repository. Our self-generated substitutes caused reproducible crashes whose cause could not be determined from the code alone. Not all figures could be unambiguously attributed to a benchmark script either, so the configuration behind some published curves remained unclear. After contacting the authors, we received the original datasets, which resolved the crashes and reduced the discrepancies. Absolute throughput still deviates from the published values in places by orders of magnitude. We cannot attribute the remaining gap to a single cause, as differing hardware, a newer toolchain and possibly different script versions all plausibly contribute. Qualitative trends such as relative ordering of structures and curve shapes are mostly preserved, as shown in Figure 2 and Figure 1.

IV. IMPLEMENTATION OF A KEY-VALUE STORE BASED ON B-TREES

Based on `libbtree`, we built a Redis-compatible key-value store. The first step was implementing the Redis Serialization Protocol (RESP, Version 2), a text-based encoding for strings, integers and arrays, sent over TCP via Boost.Asio. The server parses incoming commands and dispatches them to `libbtree`. Implemented commands include GET, SET, MGET, MSET, DEL, EXISTS, INCR/DECR, INFO, and an adaptation of sorted-set operations (ZADD, ZRANGE). Since

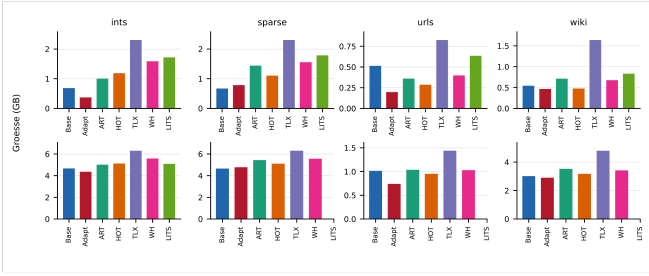


Fig. 1: Absolute memory usage. Top: original paper. Bottom: our results.

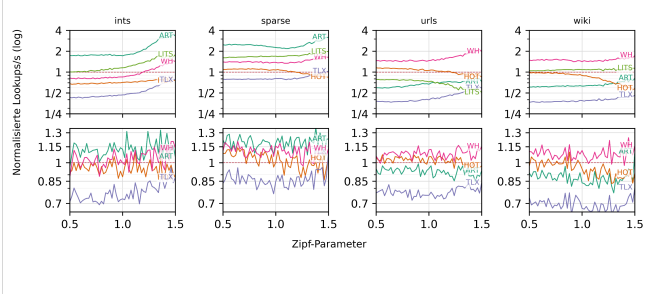


Fig. 2: Minserts normalized to B-Trees. Top: original paper. Bottom: our results.

the protocol is implemented faithfully, the resulting server [2] can be used as a drop-in replacement for Redis.

To compare both systems, YCSB2 was adapted to use a Redis client instead of direct B-tree operations, so that an identical load can be directed at either server over the local network. All measurements use Workload C (100% reads, Zipf-distributed) with batched MGET and MSET commands of 10 000 operations each to minimize network overhead.

Figure 3 shows read throughput by database size. The B-tree store becomes competitive at around 500 000 keys. Beyond 2.6 million keys, Redis’s throughput flattens due to $O(1)$ hash lookups while B-tree throughput continues to grow on an $O(\log n)$ curve.

A structural advantage of B^+ -trees is their ordered key storage, which allows range scans without additional data structures. Redis can only provide this through sorted sets, which maintain a separate index. Figure 4 shows that with small key counts both systems perform similarly, but Redis’s scan duration grows much more steeply. At around 3 million keys, Redis scans take roughly twice as long, and this gap does not close as the database grows further.

V. CONCLUSION

Optimized B^+ -trees are a solid foundation for competitive in-memory key-value stores. Our Redis-compatible implementation confirms this: it reaches competitive read throughput on sufficiently large datasets, and its ordered key storage gives it a structural advantage on range scans that a hash-based store can only match by maintaining an additional index.

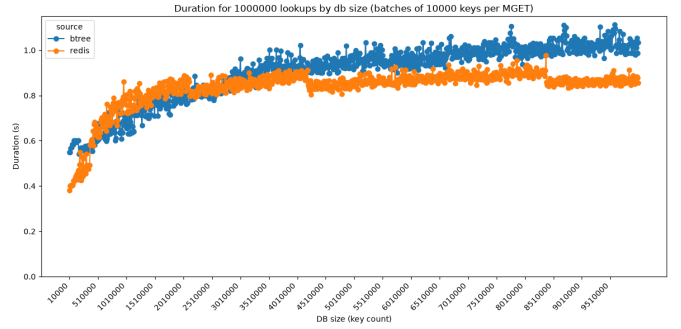


Fig. 3: YCSB2 Workload C: Throughput B-Tree vs. Redis by database size.

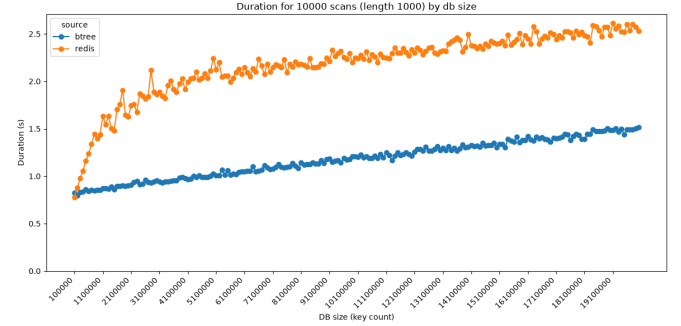


Fig. 4: Throughput of range scans: B-Tree vs. Redis by database size.

The reproduction was the more instructive part of the project. The published code alone is not sufficient for reproducing the benchmarking results. The datasets, the machine configuration and an unambiguous mapping from figures to scripts were needed as well, and only the first of these could be recovered, by contacting the authors directly. In a field where performance claims are the primary contribution, experimental artifacts deserve the same care in publication as the source code.

REFERENCES

- [1] M. Müller, L. Benson, and V. Leis, “B-trees are back: Engineering fast and pageable node layouts,” *Proceedings of the ACM on Management of Data*, vol. 3, no. 1, pp. 1–26, 2025.
- [2] J. Gruhler, M. Bellm, A. Rico Martin, and M. Raba, “btrees_redis: Redis-compatible in-memory key-value store based on optimized B^+ -trees,” https://github.com/kantholz44mm/btree_redis, 2026.